

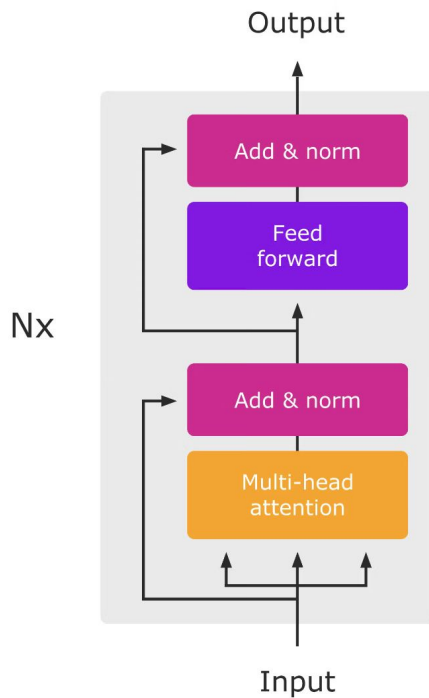
Audio Language Models

(Speech Understanding)

Grinberg Petr
HSE 2025

Recap: LLMs

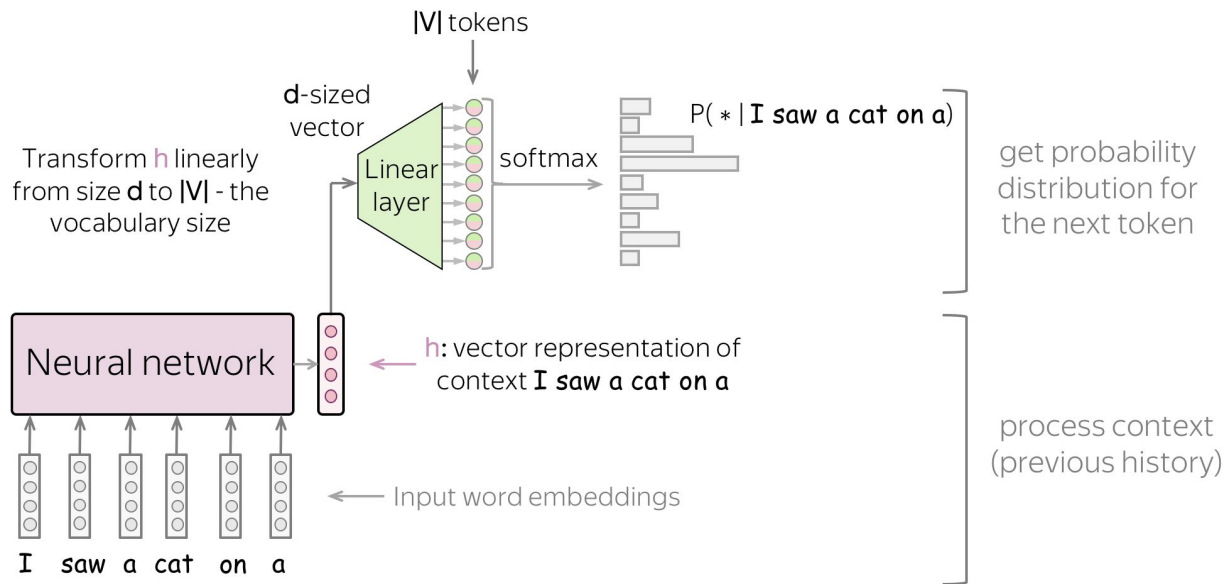
Transformer Decoder



Recap: LLMs

Language Modelling

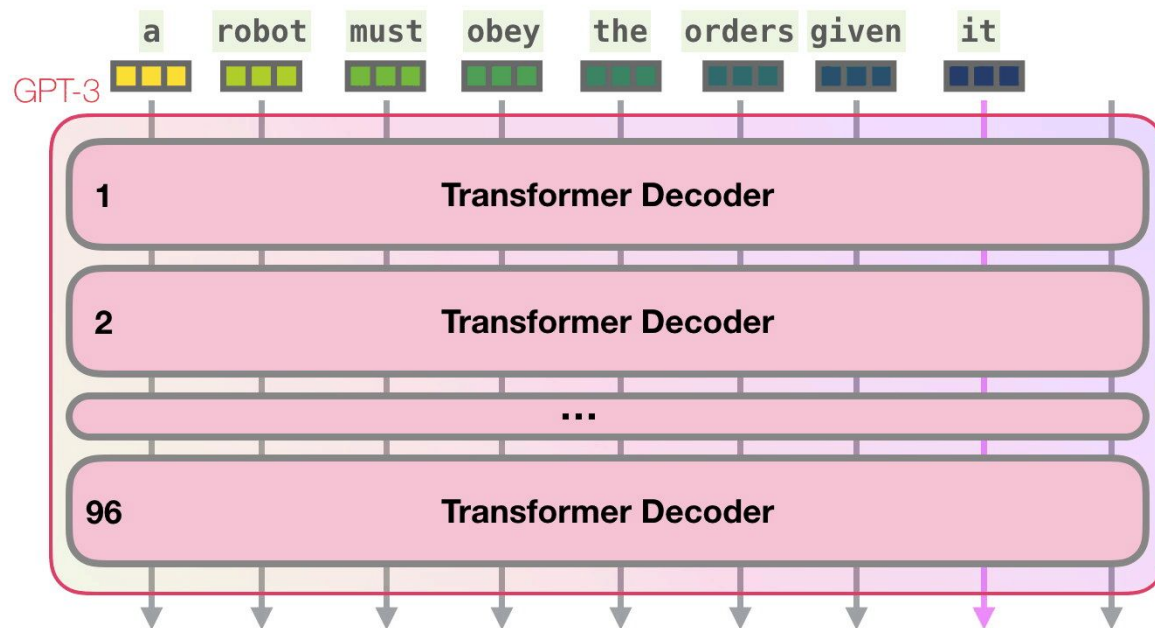
$$P(y_1, y_2, \dots, y_n) = P(y_1) \cdot P(y_2|y_1) \cdot P(y_3|y_1, y_2) \cdot \dots \cdot P(y_n|y_1, \dots, y_{n-1}) = \prod_{t=1}^n P(y_t|y_{<t}).$$



**Cross-Entropy
over vocabulary
(classification)**

Recap: LLMs

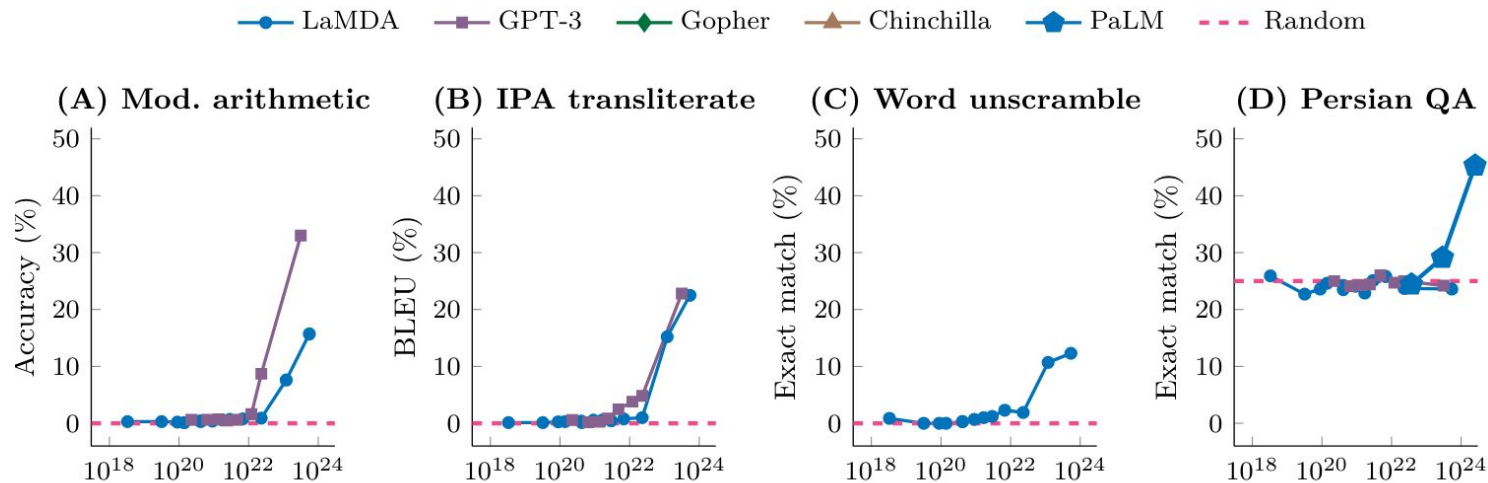
Decoder-Only



Recap: LLMs

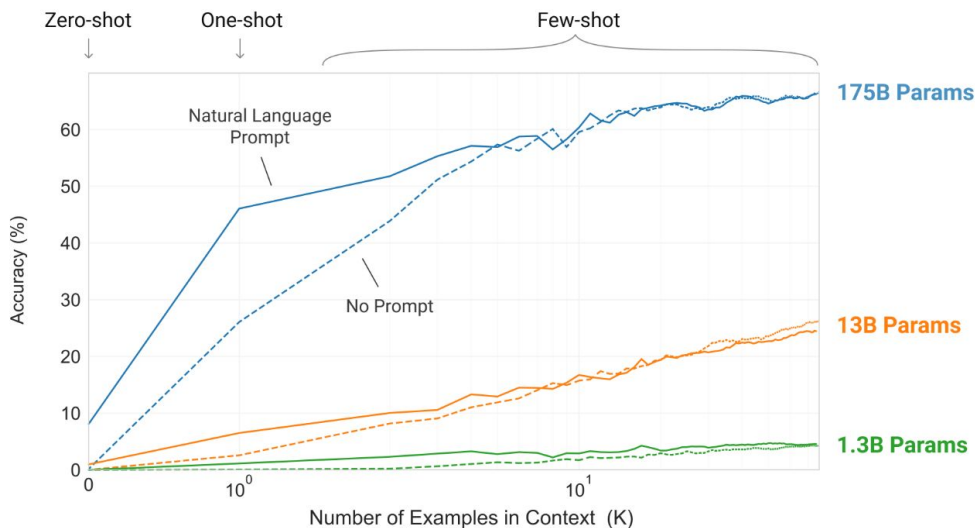
More layers, more data – emergent abilities

Emergence is when quantitative changes in a system result in qualitative changes in behavior.



Recap: LLMs

More layers, more data – emergent abilities



Recap: LLMs

Supervised Fine-Tuning (SFT), a.k.a. Instruction-tuning + Preference Optimization

Step 1

Collect demonstration data, and train a supervised policy.

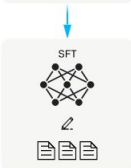
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

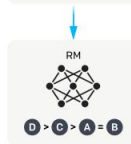
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



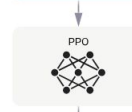
Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



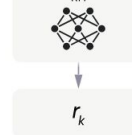
The policy generates an output.



The reward model calculates a reward for the output.

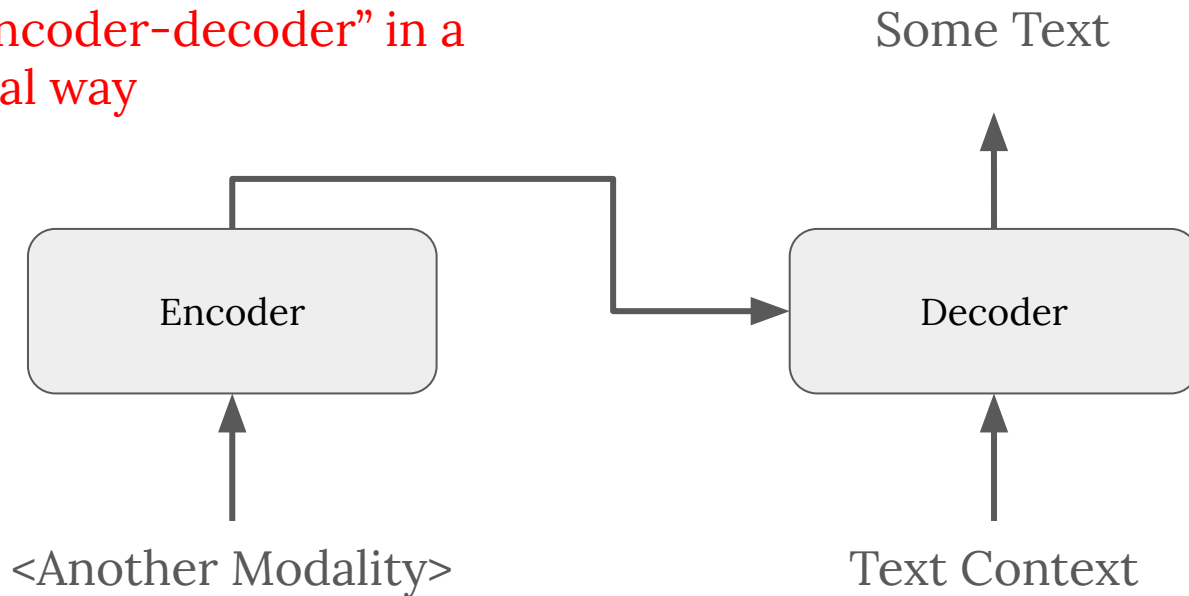


The reward is used to update the policy using PPO.

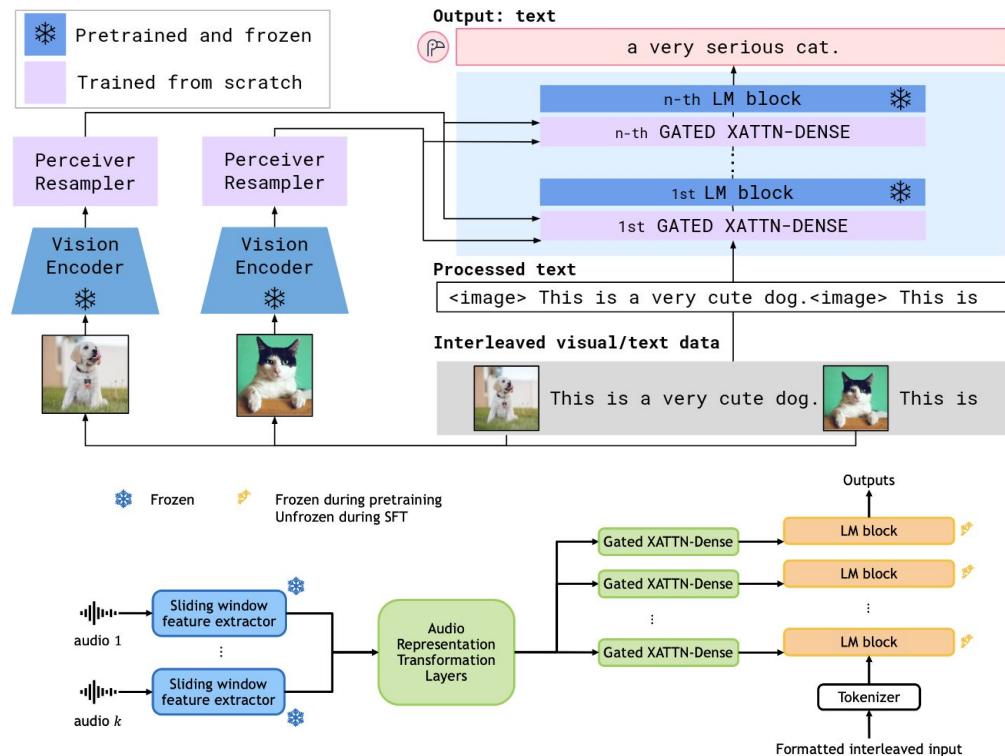


Multimodal LMs – Encoder-Decoder Style

Not “encoder-decoder” in a classical way



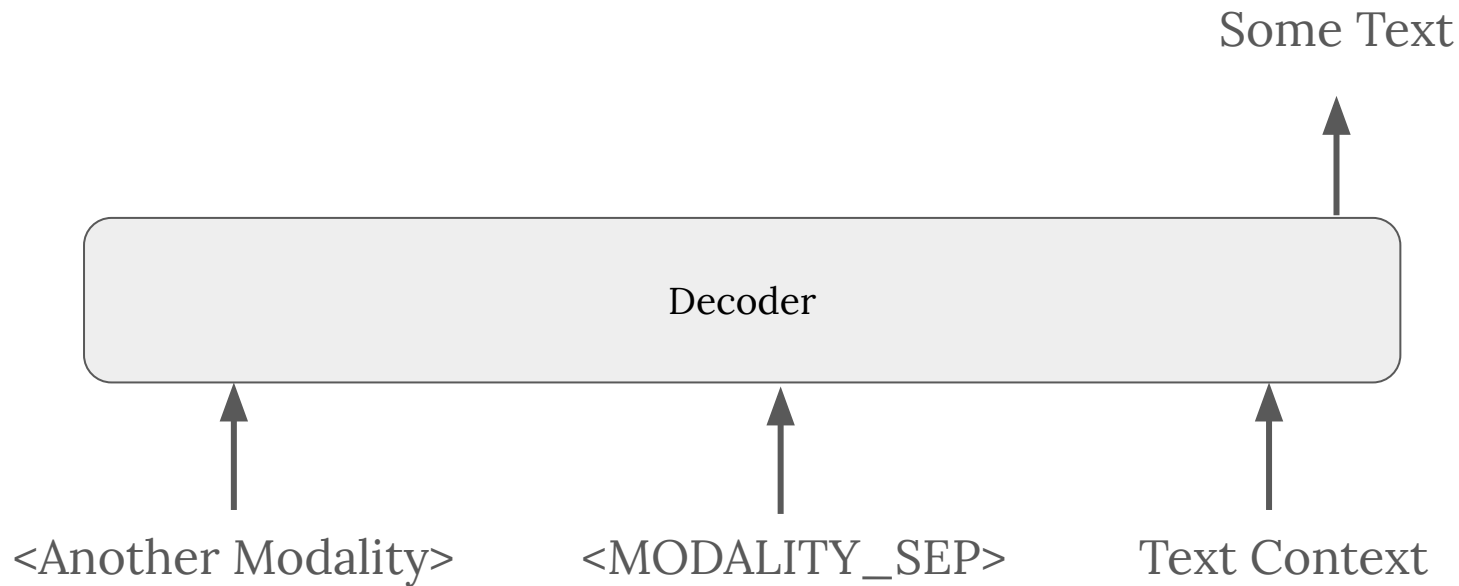
Multimodal LMs – Encoder-Decoder Style



[Flamingo: a Visual Language Model for Few-Shot Learning \(NIPS 22\)](#)

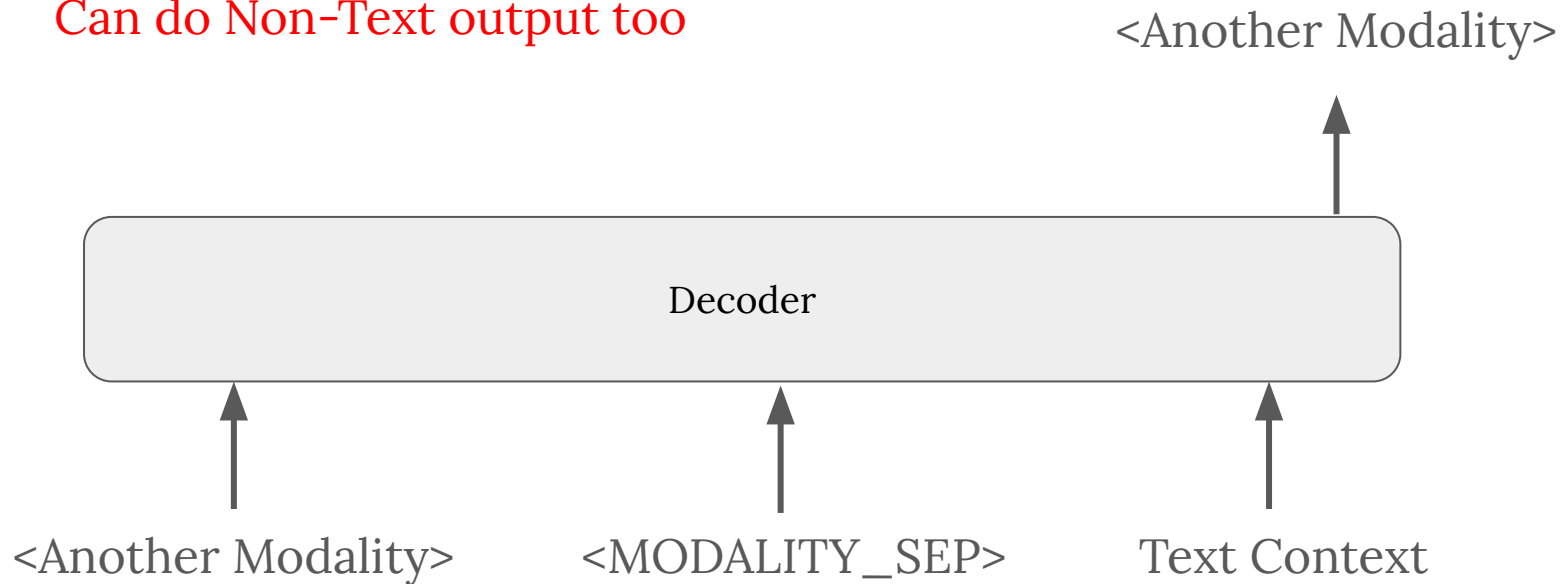
[Audio Flamingo: A Novel Audio Language Model with Few-Shot Learning and Dialogue Abilities \(ICML 2024\)](#)

Multimodal LMs – Prefix-Decoder Style

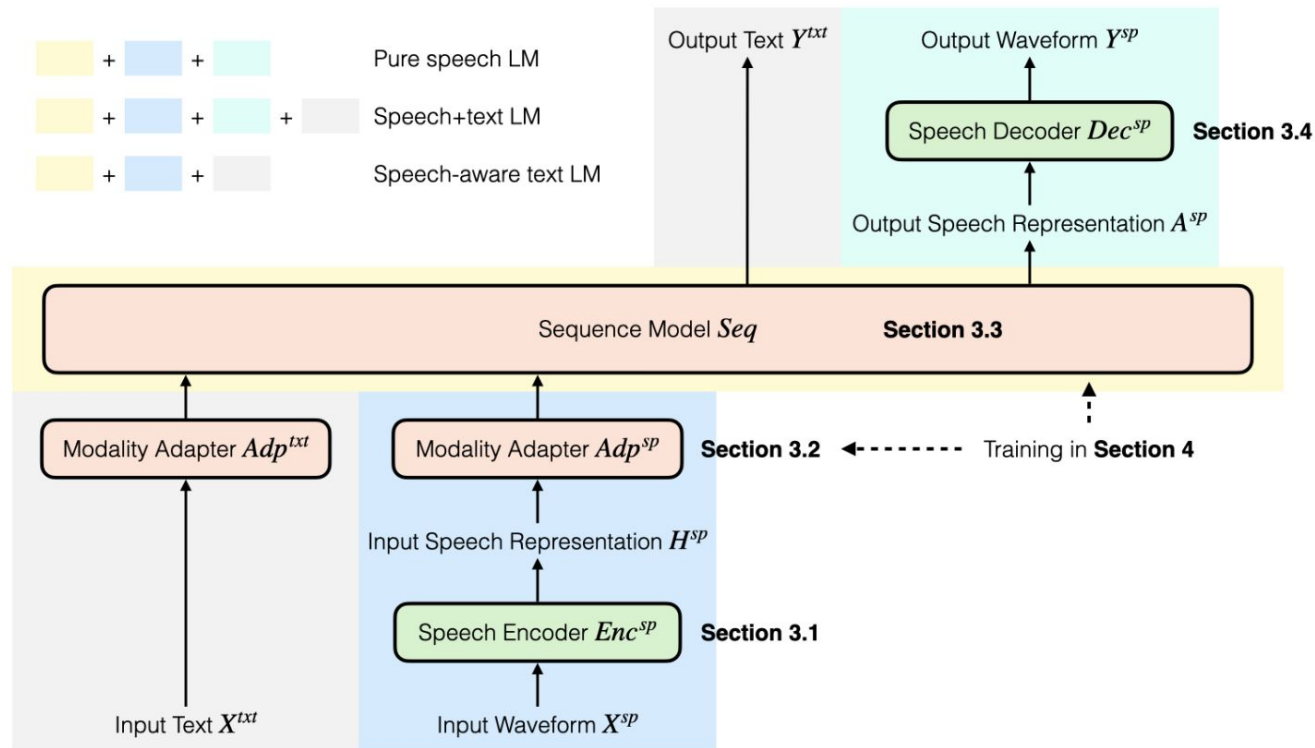


Multimodal LMs – Prefix-Decoder Style

Can do Non-Text output too



Multimodal LMs – Prefix-Decoder Style



Tokens types

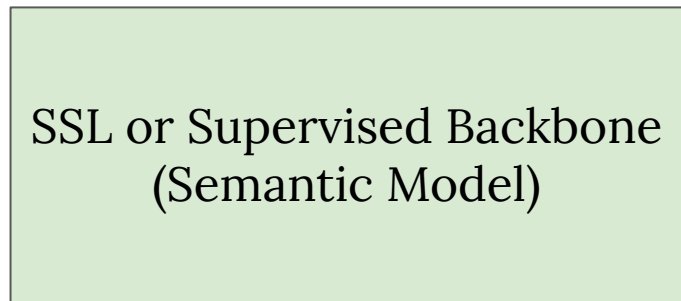
Information type:

- Semantic tokens – care about the content
 - Prosody
 - Paralinguistics
 - Speaker info
 - Words/Phonemes
 - Etc.
- Acoustic tokens – care how the audio sounds
 - Rhythm
 - Pitch, Loudness, Noisiness
 - Timbre

Data type:

- Discrete (like text) – sequence $a[n] = [1, 100, 320, 104, \dots]$
- Continuous (like output of `nn.Embedding` when discrete tokens are passed there) – embeddings in the $\mathbb{R}^d$, where d is the LLM input embedding size

Tokens types



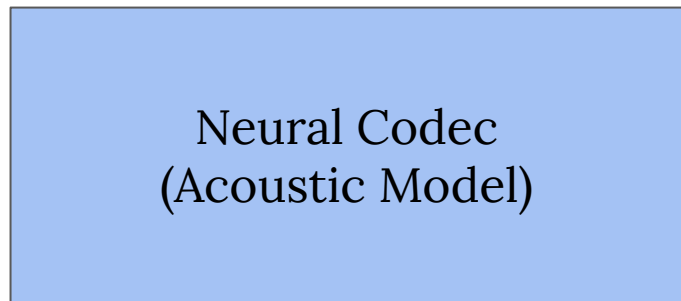
embeddings	
2.41; 1.52; 100...	11.1
30.1; 11.12; 0...	21.9
4.55; 2.31; 20...	12.7
9.13; 10.43; 30...	33.1
5.11; 3.90; 99...	0.19

clusters



units

2
43
10
9
9



1
10
32
32
5

Codecs are also
self-supervised but
the approach is
different and more
acoustic focused

Speech Generation & Speech Output Later In the Course

Speech Understanding and IF
(Speech + Text) $\rightarrow$ Text

Speech Encoders

What do we want?

A robust Audio LLM that can generalize to:

- (Optional) Different Audio Types (Speech, Event, Music)
- Different Data Conditions (Clean, Noisy, Reverberated, Bad Mic, etc.)
- Different Sources
 - Different Gender
 - Different Language
 - Different Accent
 - Different Emotions
 - Different Events
 - Different Instruments
 - Different Pitches
 - Different Speech

We need a robust audio-to-LLM-input (tokens or embeddings) encoder

What do we want?

A robust Audio LLM that can generalize to:

- (Optional) Different Audio Types (Speech, Event, Music)
- Different Data Conditions (Clean, Noisy, Reverberated, Bad Mic, etc.)
- Different Sources
 - Different Gender
 - Different Language
 - Different Accent
 - Different Emotions
 - Different Events
 - Different Instruments
 - Different Pitches
 - Different Speech

Most likely we also want to
have text-based
generalization

We need a robust audio-to-LLM-input (tokens or embeddings) encoder

What do we want?

A robust Audio LLM that can generalize to:

- (Optional) Different Audio Types (Speech, Event, Music)
- Different Data Conditions (Clean, Noisy, Reverberated, Bad Mic, etc.)
- Different Sources
 - Different Gender
 - Different Language
 - Different Accent
 - Different Emotions
 - Different Events
 - Different Instruments
 - Different Pitches
 - Different Speech

And generalization to
different tasks/prompts

We need a robust audio-to-LLM-input (tokens or embeddings) encoder

Achieving Generalization

We need some model to convert audio into tokens and achieve generalization

What if we train from scratch?*

- Training full model (including LLM) -> lack of audio-text data and want to solve text-only tasks -> will train on text anyway, no need for scratch
- Training Text LLM with random init of encoder -> need target outputs, might not have enough diverse labels -> will lack generalization and will perform poorly from an extremely noisy start

Training from scratch is **redundant** and harms convergence and performance

*Some literature tries it though: see [survey](#)

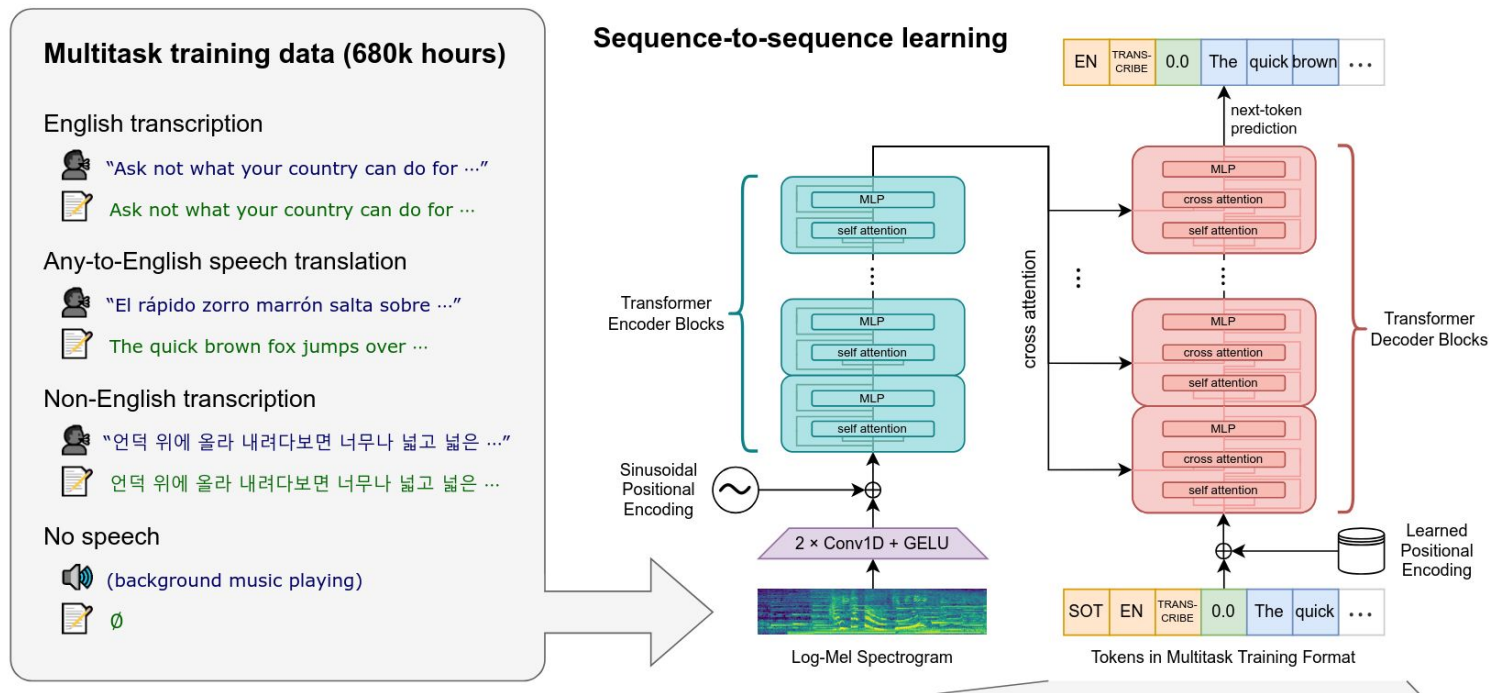
Achieving Generalization

We can for sure reuse the Text LLM. Pretraining audio encoder helps to have a nice start for the Audio LLM and achieve better generalization.

Pretrained encoders:

- ASR backbones -> the most common labeled data, very good in understanding speech content and can generalize to speech understanding
- SSL backbones -> do not need labels at all and can be pretrained on larger corpora and different audio types -> result in general representations that are suitable for many downstream tasks, not only ASR

Recall Whisper – Can use its Encoder as Audio Encoder



Why Whisper?

Even though it is pretrained for ASR/Translation, Whisper has two benefits

- It is already designed for Autoregressive modeling
- Extremely large training corpora (OpenAI model): 680k hours, which gives the model diverse knowledge
 - Many SSL models are trained on <100k hours (but there are those with more hours, e.g. 4.5M hours)
- Due to ASR task it is much better in capturing speech
 - Adapters on top of SSL backbones would need to learn how to do ASR, on Whisper it is easy
 - But Whisper is probably worse in tasks not related to speech content, especially when the input is not speech (events or music). That's where other backbones (pretrained on even larger corpora) might be better

Self-Supervised Models

No labels -> Much more data available

But we need some loss function to train the models on

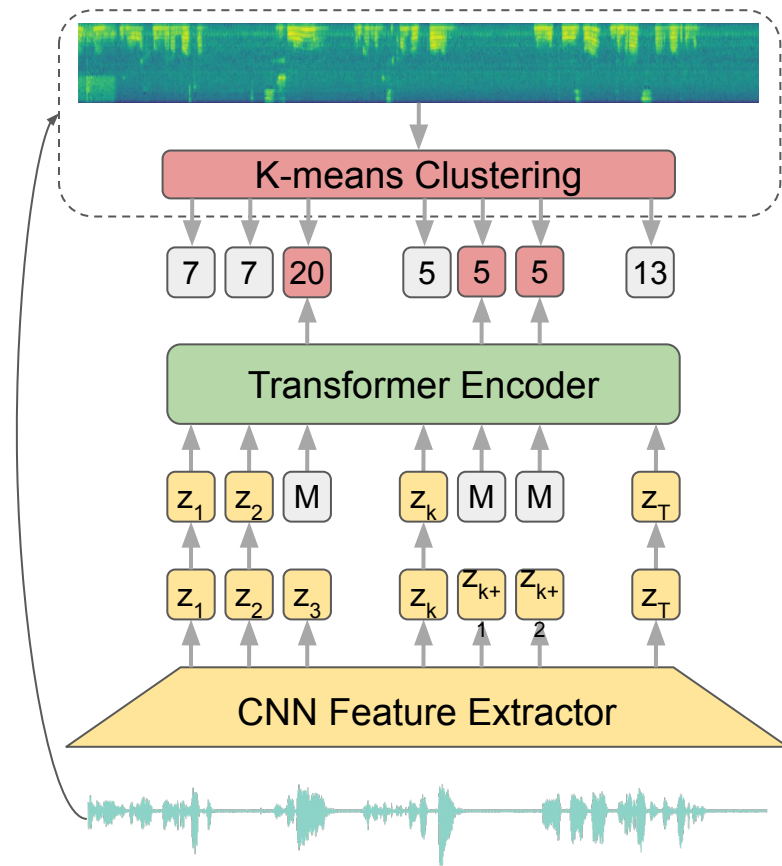
Example approaches:

- Masking -> Try to predict masked part of the input
- Contrastive learning -> Compare input with different inputs
- Autoencoders -> Recover from compressed representation

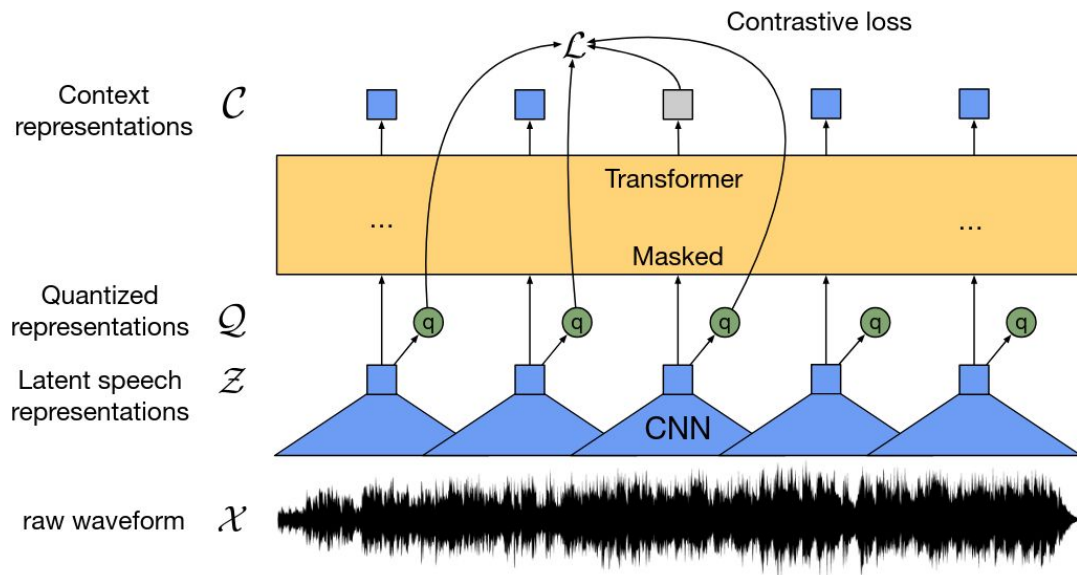
HuBERT

Masked modeling using K-means on features:

1. Start with MFCC for audio
2. Repeat N times with intermediate representations of HuBERT itself

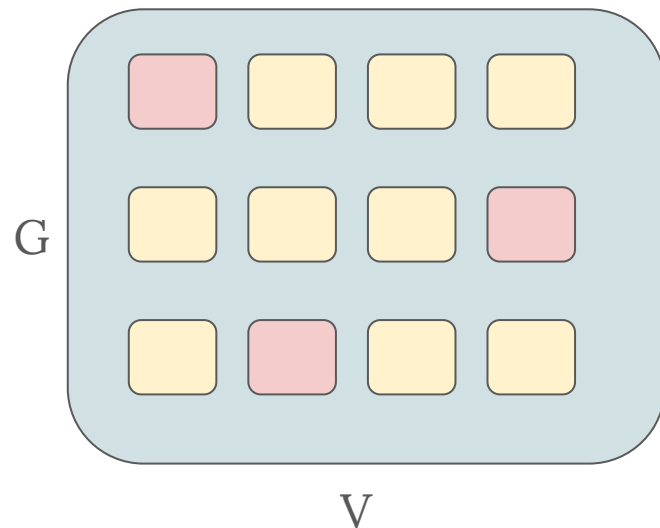


Wav2Vec 2.0

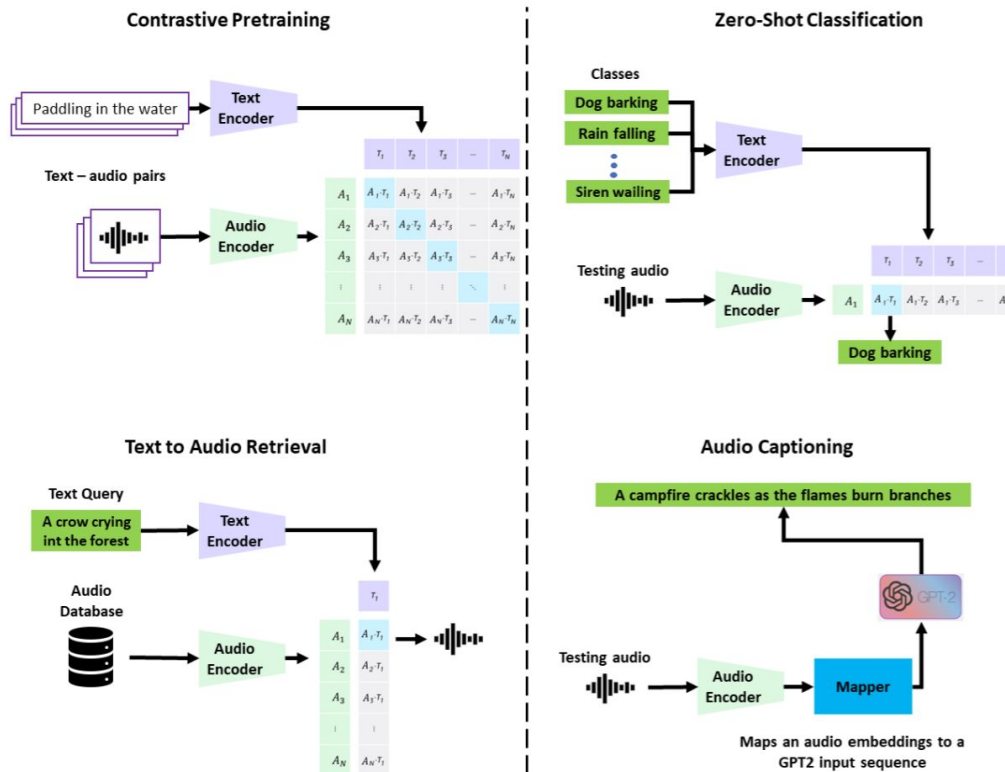


$$\Pr \left[\arg \max_j (l_j + g_j) = i \right] = \frac{e^{l_i}}{\sum_k e^{l_k}}$$

$$y_i = \frac{\exp((l_i + g_i)/\tau)}{\sum_j \exp((l_j + g_j)/\tau)}$$



CLIP for Audio? -> CLAP



Not used widely
in Audio LMs

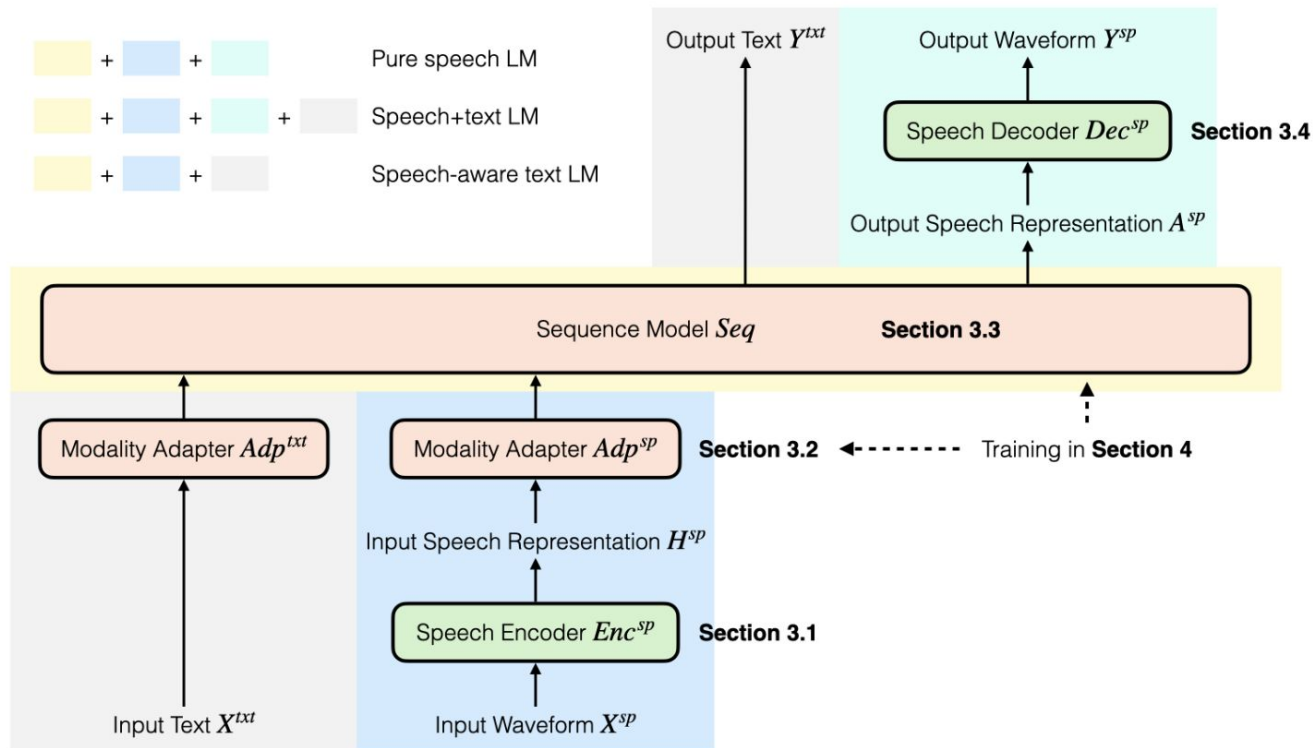
Probably due to
not large enough
training corpora

[Clap learning audio concepts from natural language supervision \(ICASSP 2023\)](#)

[Natural Language Supervision for General-Purpose Audio Representations \(arXiv 2023\)](#)

Audio LMs

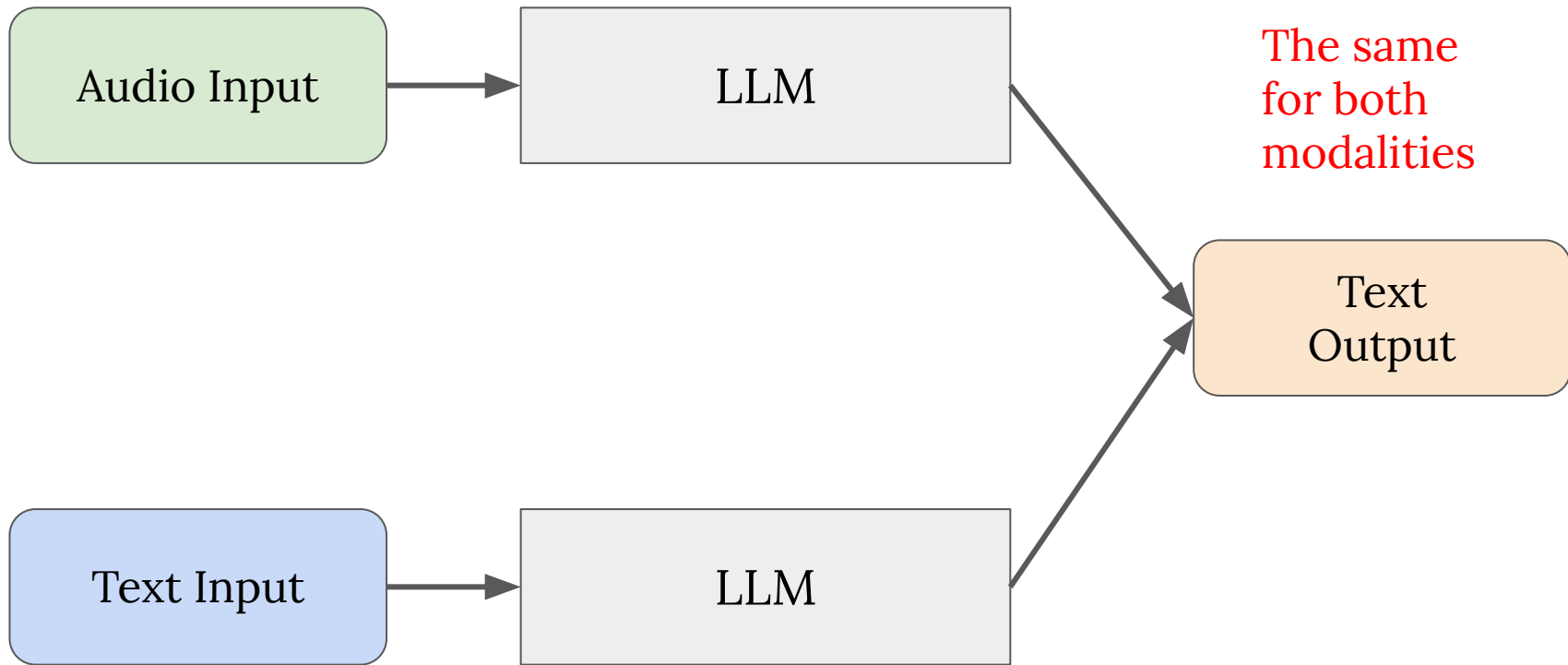
General view of Audio LMs:



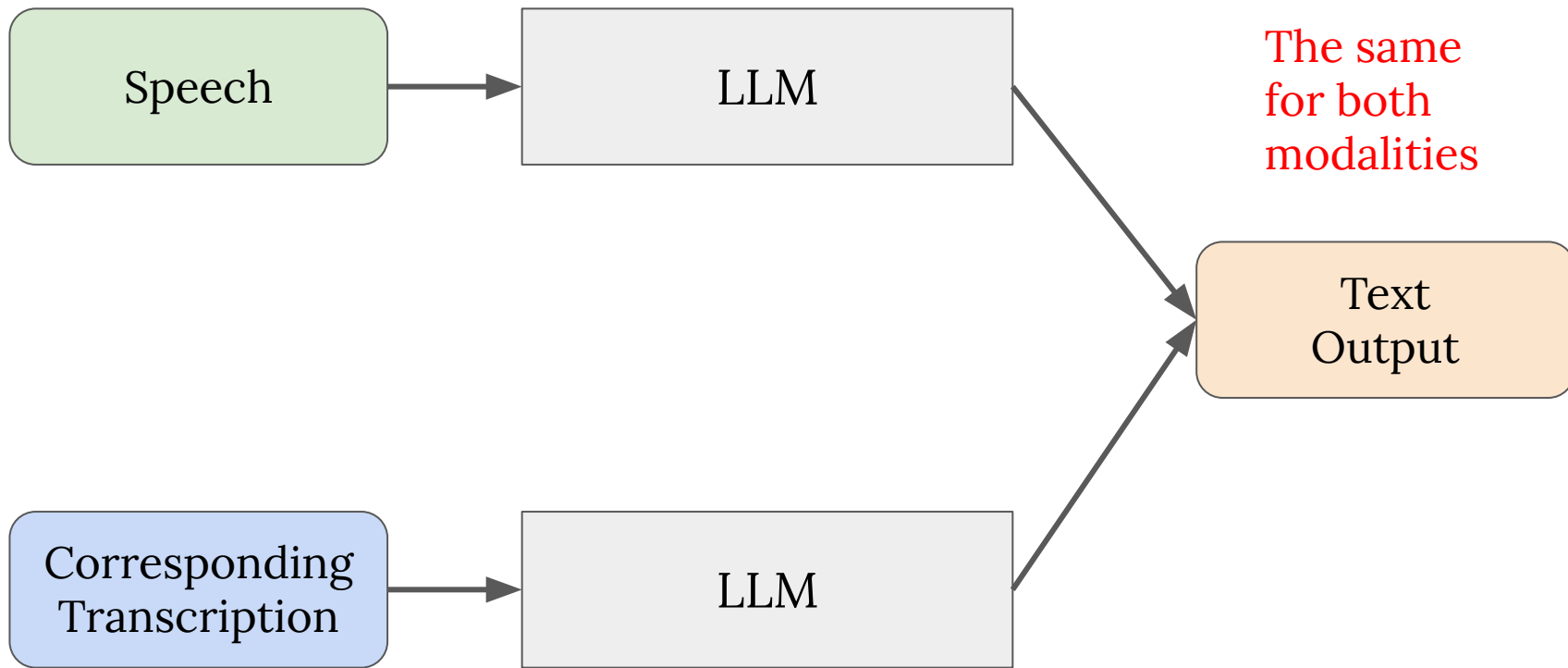
Frozen LLM vs Fine-tuning

- Full Fine-tuning -> expensive
- Adapters (LoRA) -> cheaper
 - Lack of audio data or prompt diversity -> Continuous forgetting issue
- Fully Frozen LLM -> Cheap, preserve text quality and generalization
 - May have limited audio understanding capabilities

Audio-Text Alignment – Modal-Invariance Trick

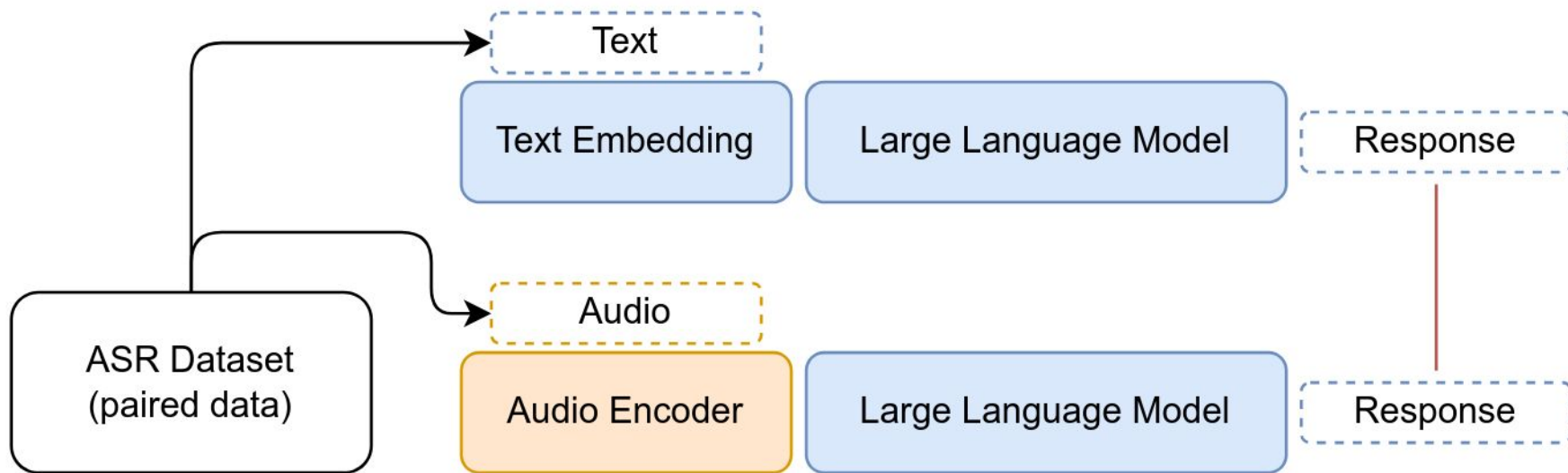


Audio-Text Alignment – Modal-Invariance Trick



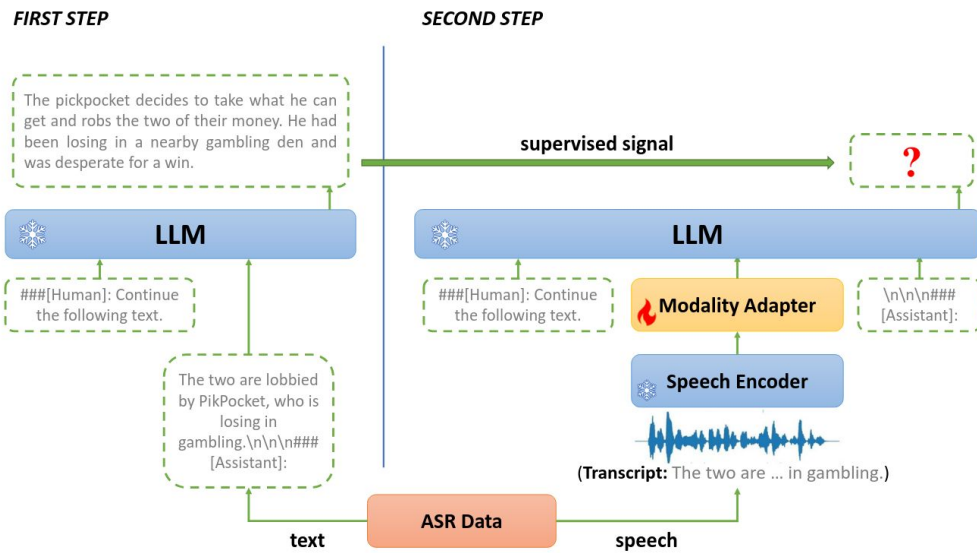
Audio-Text Alignment – Modal-Invariance Trick

Transcription as prompt/instruction

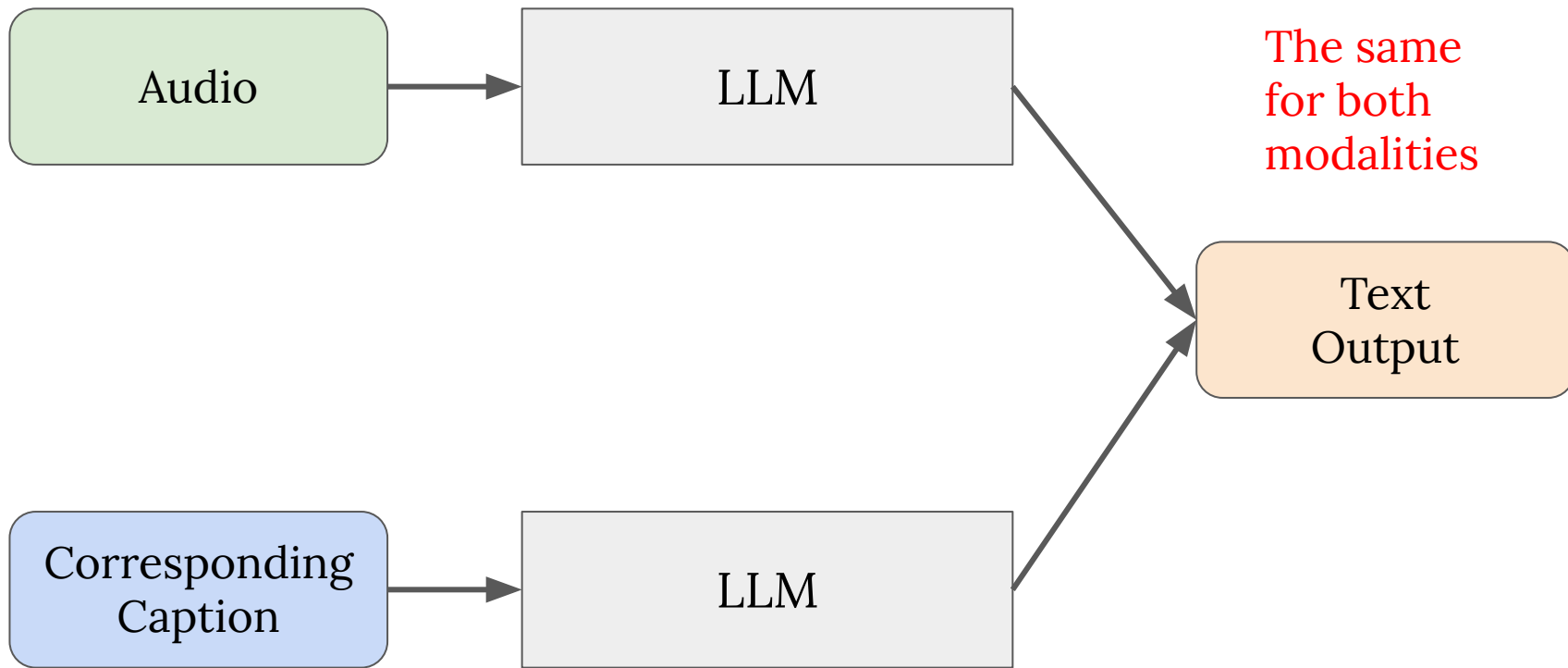


Audio-Text Alignment – Modal-Invariance Trick

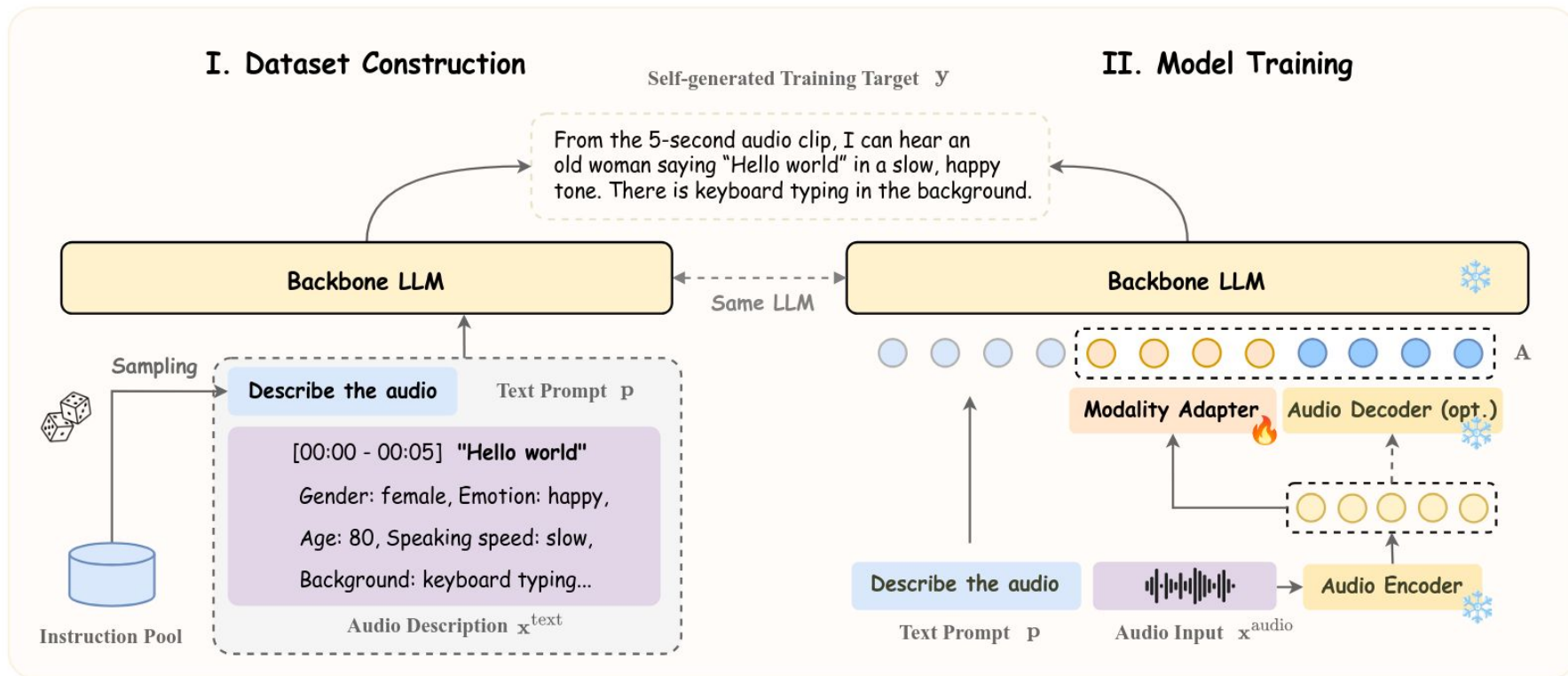
Transcription with “Generate Continuation” prompt



Audio-Text Alignment – Modal-Invariance Trick



Audio-Text Alignment – Modal-Invariance Trick



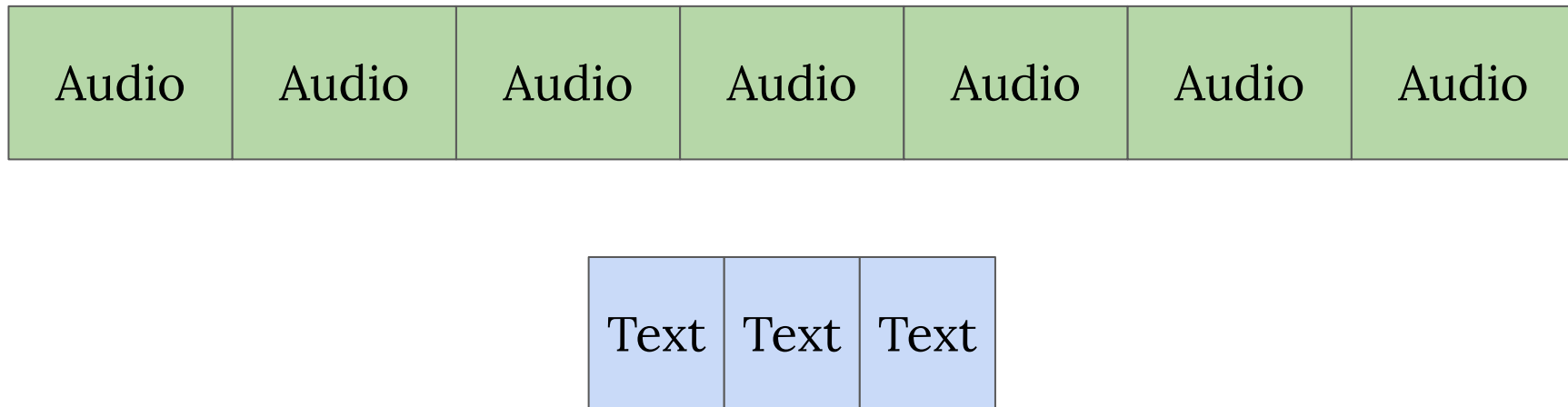
Other dataset creation approaches

- TTS on top of text datasets -> synthesize speech
- Creating captions from labels on supervised data (e.g. “The speaker sounds {emotion_class}”)
- Distillation from existing
 - Audio LMs (directly from audio)
 - Text LLMs (from captions)

Dimensionality problem

Audio: 16000 points per second -> 40-100Hz after Spectrogram/Encoder

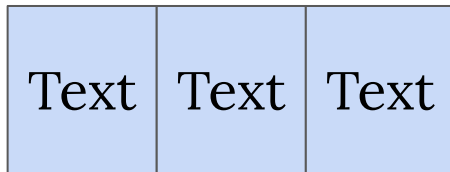
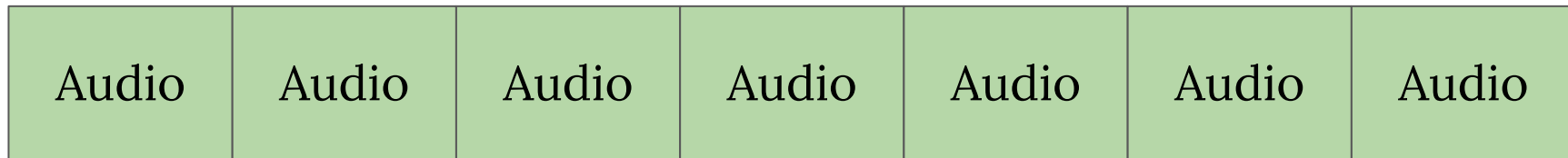
Text (transcription): a couple of words “per second” (+ BPE)



Dimensionality problem

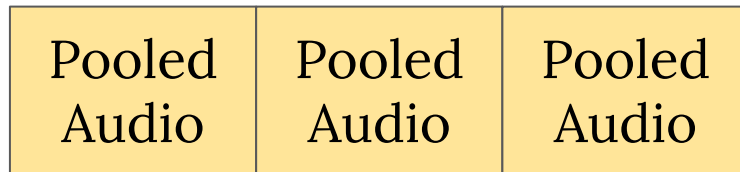
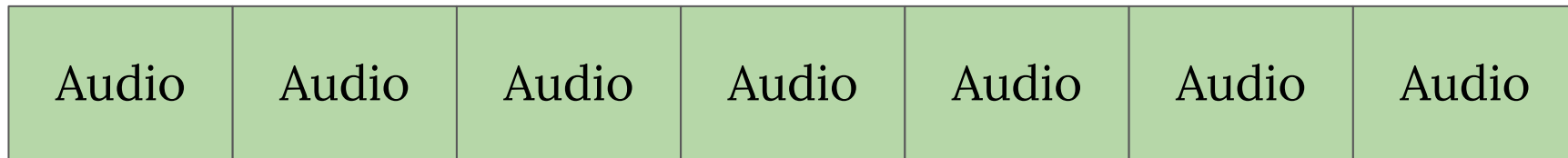
Working with audio is expensive

> 30s of audio input -> especially hard

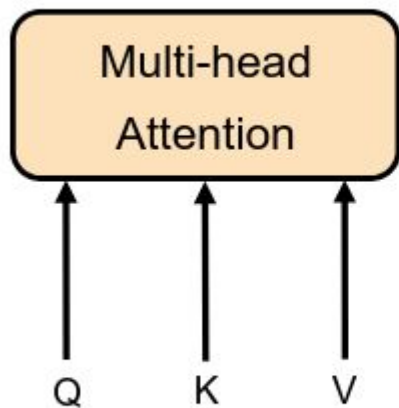


Subsampling

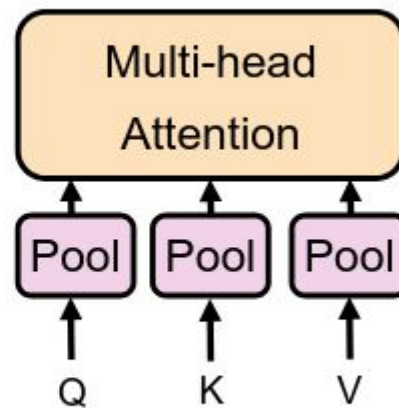
- Average pooling
- Convolutions
- Other tricks (see below)



M-Adapter – Pooling inside attention

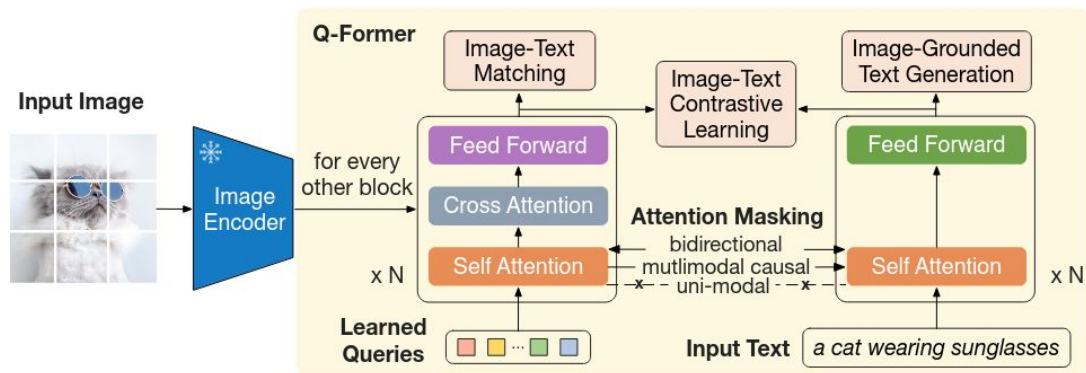


(a)



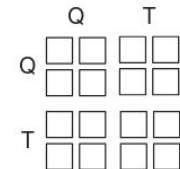
(b) (ours)

Q-Former – Fixed number of Transformer outputs



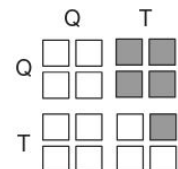
Q: query token positions; **T**: text token positions.

■ masked □ unmasked



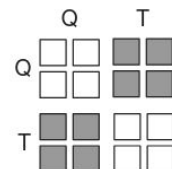
Bi-directional Self-Attention Mask

Image-Text Matching



Multi-modal Causal Self-Attention Mask

Image-Grounded Text Generation



Uni-modal Self-Attention Mask

Image-Text Contrastive Learning

Q-Former – Fixed number of Transformer outputs

Q-Former variations:

- Fixed Number of Queries -> Any sequence to exactly N tokens (e.g. DESTA)
- Window Q-Former -> Each window to exactly N tokens (e.g. [SALMONN](#))
- Dynamic Q-Former -> Window Q-former with varying length window

CTC-Compressor

Align audio tokens with text using ASR transcriptions:

Each hidden unit is matched with a letter:

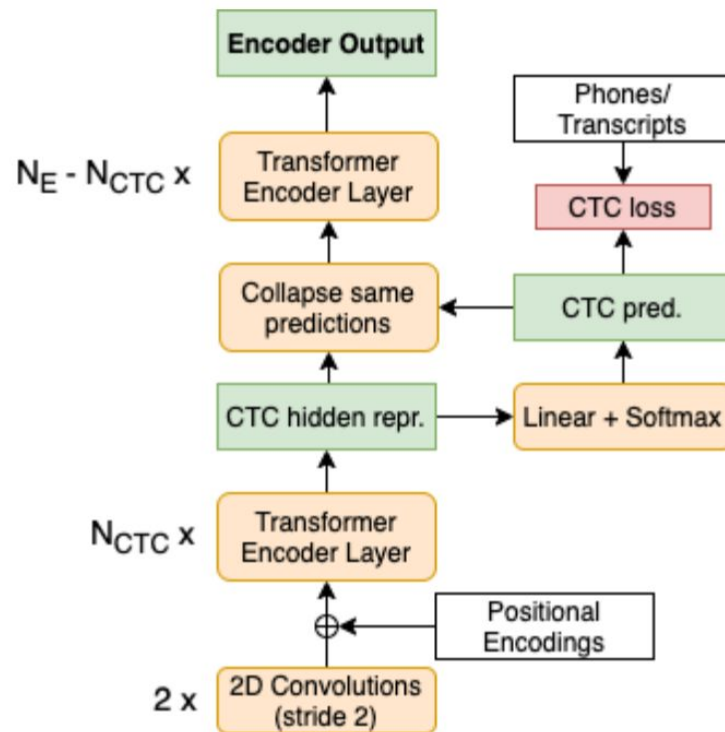
$h_1 \rightarrow \text{"c"}$

$h_2 \rightarrow \text{"c"}$

$h_3 \rightarrow \text{"a"}$

$h_4 \rightarrow \text{"t"}$

Join same letters $\rightarrow [(h_1+h_2), h_3, h_4]$



CTC-Compressor

Joining:

- Average
- Weighted Average (proba $\rightarrow$ weight)
- Window Q-Former!!!

